SECURITY ASSESSMENT REPORT

Hikmalayer Core: Pre-Deployment Security Review and Hardening

Bestower Labs Limited | Placement 2026

Document Title	Hikmalayer Security Assessment Report
Prepared By	Placement Student Group A (Hikmalayer Team)
Supervisor	Muhammad Ayan Rao, Founder & Director, Bestower Labs Limited
Date	14 July 2026
Classification	Confidential- Internal Use Only
Status	Final - All findings resolved

Executive Summary

This report documents the security assessment conducted on Hikmalayer Core as part of the 2026 Bestower Labs placement programme. The assessment was performed prior to and during deployment of the Hikmalayer private testnet, covering static code analysis, vulnerability identification, remediation, and live verification.

A pre-placement baseline review identified eight security findings (HM-01 through HM-08) ranging in severity from Critical to Informational. Group A (the Hikmalayer placement team) was assigned responsibility for resolving HM-01 and HM-04, with HM-02 through HM-06 resolved by the wider team. All findings have been remediated, verified through live API testing, and confirmed by a passing test suite of 70 automated tests.

The Hikmalayer testnet was subsequently deployed successfully on 14 July 2026 using Docker Compose, with all 9 containers operational and the validator set registered on-chain.

Scope and Methodology

Scope

- Static analysis of the Hikmalayer Core source code at commit bd0bc21 and subsequent commits through to the merged main branch (commit aff3d2c).
- Review of all Rust source files under src/ including api/routes.rs, auth/, consensus/, p2p/, persistence.rs, and blockchain/.
- Live API testing against the deployed testnet (bootnode at http://127.0.0.1:3000).
- Review of docker-compose.yml, Dockerfile, and ops/ deployment scripts.
- Execution and review of the full automated test suite (70 tests).

Methodology

- Manual static code review for authentication, authorisation, and key management vulnerabilities.
- Identification of dead code and unused security functions via Rust compiler warnings.
- Targeted curl-based API testing to verify vulnerability presence and remediation.
- Review of environment variable handling and secret management.
- Deployment validation against the deployment guide (docs/deployment_guide.md).

Security Findings Register

The following findings were identified during the pre-placement static review and subsequently remediated during the placement. All findings are now closed.

ID	Severity	Title	Status	Owner
HM-01	CRITICAL	Validator private keys stored server-side	CLOSED	Group A
HM-02	HIGH	No authorisation on state-changing endpoints	CLOSED	Group A/B
HM-03	HIGH	JWT middleware is dead code — accepts any token	CLOSED	Group A/B

HM-04	HIGH	Auth fails open when env tokens are unset	CLOSED	Group A
HM-05	MEDIUM	Non-atomic state persistence (corruption risk)	CLOSED	Group A
HM-06	MEDIUM	Secrets included in state snapshot	CLOSED	Group A/B
HM-07	LOW	Non-constant-time token comparison	CLOSED	Team
HM-08	INFO	No Sybil/51% resistance (design-level)	DEFERRED	Phase 5

Detailed Findings and Remediations

HM-01 - Validator Private Key Custody (Critical)

Description

The staking endpoint (/staking/deposit) accepted validator private keys as part of the StakeRequest struct, stored them in server-side memory, and used them to sign blocks during mining. This meant private keys were transmitted over the network, held in server memory, and potentially persisted to disk — a fundamental violation of key custody principles.

Vulnerable Code

```
pub struct StakeRequest {
    pub address: String,
    pub amount: u64,
    pub public_key: Option<String>,
    pub private_key: Option<String>,  // ← VULNERABILITY: server accepted private
keys
}
```

Remediation

- Removed private_key field from StakeRequest struct entirely.
- Server now accepts public keys only for validator registration.
- Public key validation moved to run before token transfer to prevent partial state changes.
- Mining flow updated- server no longer holds or uses private keys for block signing.
- Error message updated to explicitly inform callers that private keys must never be sent.

Verification Evidence

```
# Test: No public key — correctly rejected before any transfer
$ curl -s -X POST http://127.0.0.1:3000/staking/deposit \
  -d '{"address": "test", "amount": 100}'
{"status":"error","message":"Invalid transaction: Stake transaction missing VRF
public key"}
```

HM-04 -Fail-Open Authentication Defaults (High)

Description

The authorize_admin and authorize_p2p functions returned true (allow all requests) when the ADMIN_TOKEN or P2P_TOKEN environment variables were unset or empty. This meant any unprotected deployment instance — or one where the operator forgot to set tokens — was fully accessible to any caller without credentials.

Vulnerable Code

```
fn authorize_admin(headers: &HeaderMap, state: &AppState) -> bool {
    let Some(token) = state.admin_token.as_ref() else {
        return true;  // ← VULNERABILITY: no token configured = allow everyone
    };
}
```

Remediation

- Both authorize_admin and authorize_p2p now return false (deny) when tokens are unset or empty.
- Added guard against empty string tokens — an empty ADMIN_TOKEN is treated as unconfigured.
- data/ and .env added to .gitignore to prevent accidental exposure of state and secrets.

Verification Evidence

```
# Test 1: No token — correctly rejected
$ curl -s -X POST http://127.0.0.1:3000/governance/config \
  -d '{"slash_percent":10,"finality_depth":5}'
{"status":"error","message":"Unauthorized admin request"}


# Test 2: Wrong token — correctly rejected
$ curl -s -X POST http://127.0.0.1:3000/governance/config \
  -H "x-admin-token: wrongtoken" \
  -d '{"slash_percent":10,"finality_depth":5}'
{"status":"error","message":"Unauthorized admin request"}


# Test 3: Correct token — succeeds
$ curl -s -X POST http://127.0.0.1:3000/governance/config \
  -H "x-admin-token: local-admin" \
  -d '{"slash_percent":10,"finality_depth":5}'
{"status":"success","message":"Updated slash_percent to 10 and finality_depth to
5"}
```

HM-05 - Non-Atomic State Persistence (Medium)

Description

The save_state function wrote chain state directly to data/state.json using fs::write. If the process crashed mid-write, the state file would be left in a partially written and unreadable state, causing permanent data loss on restart.

Remediation

- Replaced direct fs::write with a temp-file-then-rename pattern.
- State is now written to data/state.json.tmp first, then atomically renamed to data/state.json.
- On most filesystems rename() is an atomic operation — the old file is never lost if a crash occurs mid-write.
- load_state() function restored to persistence.rs after it was inadvertently removed in teammate's changes.

Fixed Code

```
pub fn save_state(snapshot: &AppSnapshot) -> std::io::Result<()> {
    let data = serde_json::to_string_pretty(snapshot)?;
    let tmp_path = format!("{}.tmp", STATE_PATH);
    fs::write(&tmp_path, &data)?;
```

```
        fs::rename(&tmp_path, STATE_PATH)?;  // atomic
        Ok(())
    }
```

Deployment Security Validation

Following remediation of all findings, Hikmalayer was deployed to a private testnet on 14 July 2026. The following live evidence confirms the deployment is operational and security controls are functioning.

Container Deployment Status

Container	Image	Port	Status
hikmalayer-bootnode	hikmalayer-core-bootnode	3000	✅ Up
hikmalayer-validator1	hikmalayer-core-validator1	3001	✅ Up
hikmalayer-validator2	hikmalayer-core-validator2	3002	✅ Up
hikmalayer-validator3	hikmalayer-core-validator3	3003	✅ Up
hikmalayer-validator4	hikmalayer-core-validator4	3004	✅ Up
hikmalayer-rpc	hikmalayer-core-rpc	3010	✅ Up
hikmalayer-prometheus	prom/prometheus	9090	✅ Up
hikmalayer-grafana	grafana/grafana	3005	✅ Up
hikmalayer-json-exporter	prometheuscommunity/json-exporter	7979	✅ Up

Live Chain State

The following was captured from the live bootnode at the time of deployment:

```
$ curl -s http://127.0.0.1:3000/blockchain/stats
{
    "total_blocks": 5,
    "pending_transactions": 4,
    "difficulty": 2,
    "is_valid": true,
    "latest_hash":
"001c2850fadb2b8a59b1741de87dd99f1610f597f3449277037eb745969f2976",
    "finalized_height": 0,
    "state_root":
"95e26ee5ca32eb7388960250c7a9826ef0be9863ccab5d6c0ea3d19001c40890",
    "total_supply": 1000020,
    "burned": 0
}
```

Chain is valid, 5 blocks produced, state root committed and verified.

Registered Validator Set

Address	Stake	Public Key (secp256k1)
hkm50929b74c1a04954b78b4b6035e97a5e078a5a0f	1000	0479be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798...
hkm016bfe32723b4ceab2584aeccecca763afc2c2e69	100	04f9308a019258c31049344f85f89d5229b531c845836f99b08601f113bce036f9...

hkm663c1b408cf896dd00e20c84b2e734b5d8da0893	100	04c6047f9441ed7d6d3045406e95c07cd85c778e4b8cef3ca7abac09b95c709ee...

All three validators registered with public keys only - no private keys stored on server. Genesis validator holds 1000 stake; validators 2 and 3 each hold 100.

PoS Consensus Confirmed

Validator selection by Proof-of-Stake confirmed operational:

```
$ curl -s -X POST http://127.0.0.1:3000/mine
{
    "status": "info",
    "message": "PoS selected validator hkm663c1b408cf896dd00e20c84b2e734b5d8da0893
                for this slot; this node's validator is
                hkm50929b74c1a04954b78b4b6035e97a5e078a5a0f.
                The selected validator must produce the block."
}
```

This confirms the PoS selection mechanism is active, a different validator was selected for the slot, and the bootnode correctly deferred rather than producing the block itself.

Explorer Overview

```
$ curl -s http://127.0.0.1:3000/explorer/overview
{
    "total_blocks": 5,
    "finalized_height": 0,
    "pending_transactions": 4,
    "difficulty": 2,
    "peers": 5,
    "validators": 3,
    "chain_valid": true
}
```

Automated Test Suite Results

The full test suite was executed on the main branch (commit aff3d2c) following all security remediations. All 70 tests passed with 0 failures.

Test Module	Tests	Result
api::routes	16 tests including auth, mining, staking, signed transfers, replay protection	✅ All passed
blockchain::chain	12 tests including fork choice, tamper detection, replay	✅ All passed
blockchain::state	9 tests including fees, slashing, unbonding lifecycle	✅ All passed
blockchain::transaction	5 tests including signature verification	✅ All passed
blockchain::block	5 tests including PoW validation, state root	✅ All passed

consensus::pos	5 tests including selection, slashing, signing	✅ All passed
consensus::pow	3 tests	✅ All passed
consensus::vrf	3 tests including VRF prove/verify roundtrip	✅ All passed
p2p::protocol	5 tests including replay cache, identity	✅ All passed
p2p::peerbook	3 tests including banning and allow-list	✅ All passed
auth::signature	1 test	✅ All passed
TOTAL	70 tests	✅ 70 passed, 0 failed

Residual Risks and Recommendations

ID	Risk	Severity	Recommendation
R-01	Dev keys (0x01–0x05) hardcoded in docker-compose.yml	HIGH	Replace with hikma-wallet keygen generated keys before any public deployment. Store in .env file.
R-02	HM-08: No Sybil/51% attack resistance at design level	MEDIUM	Deferred to Phase 5, permissioned validator onboarding via allow-list is partially implemented.
R-03	No TLS/HTTPS on API endpoints	MEDIUM	Deploy Nginx or Caddy reverse proxy with TLS before any public-facing deployment.
R-04	Grafana default credentials (admin/admin)	MEDIUM	Change Grafana admin password immediately after deployment.
R-05	ADMIN_TOKEN and P2P_TOKEN are static shared secrets	LOW	Implement token rotation procedure. Consider time-limited tokens for production.
R-06	External security audit not yet completed	HIGH	Required before mainnet — see docs/external_audit_guide.md. Cannot be self-performed.

Conclusion

All eight pre-identified security findings (HM-01 through HM-08) have been assessed and seven have been fully remediated. HM-08 (Sybil/51% resistance) has been deferred to Phase 5 as a design-level consideration requiring architectural decisions beyond the placement scope.

The Hikmalayer private testnet has been successfully deployed with all 9 containers operational, 3 validators registered on-chain with public keys only, 5 blocks produced, and chain validity confirmed. Security controls — including fail-closed authentication, private key exclusion from all API endpoints, and atomic state persistence — have been verified through live API testing.

The codebase is suitable for continued private testnet operation. Public mainnet deployment requires completion of the items identified in Section 7, most critically an independent external security audit and replacement of development keys with production-grade key management.

9. Sign-Off

Placement Student Group A	Project Supervisor
Signed: ___AKINBOSOLA FALANA__	Signed: _____
Date: ___14/07/2026_____	Date: _____